舆情监测分析预测系统 - 项目设计文档

一、项目背景与目标

创建一个**简化版舆情系统**，专注于三个核心功能：

功能模块	具体内容
舆情监测	定时采集新闻和社交媒体数据，监控关键词
舆情分析	情感分析、关键词提取、趋势可视化
舆情预测	基于历史数据预测舆情走向

二、系统架构设计

2.1 整体架构

```
┌──────────────────────────────────────────────────────────┐
│                      Streamlit 前端界面                   │
│  (舆情监测 | 舆情分析 | 舆情预测 | 系统设置)              │
└──────────────────────────────────────────────────────────┘
                          │
                          │
      ┌──────────────────────────┐
      │                          │
┌─────▼──────────┐      ┌──────────▼──────────┐
│  后端API层      │      │    数据处理层        │
│  (Flask/无)     │      │  (Pandas/NumPy)     │
└────────────────┘      └─────────────────────┘
      │                          │
      │                ┌──────────────────────┐
      │                │                      │
┌─────▼──────┐   ┌─────▼──────┐      ┌──────────▼──────┐
│ 数据采集    │   │ 舆情分析    │      │   舆情预测      │
│  模块       │   │   模块      │      │    模块         │
└────────────┘   └────────────┘      └────────────────┘
      │                │                      │
      └────────────────┴──────────────────────┘
                          │
              ┌──────────▼──────────┐
              │  SQLite数据库       │
              │  (数据持久化)       │
              └────────────────────┘
```

2.2 技术栈选择

层级	技术	说明

层级	技术	说明
前端	Streamlit	最简单的Python Web框架
后端	无需独立后端	Streamlit自带后端能力
数据库	SQLite	轻量级，无需安装
爬虫	Requests + BeautifulSoup	简单HTTP爬虫
数据处理	Pandas + NumPy	标准数据处理库
情感分析	SnowNLP / LLM	中文情感分析
时间序列预测	Prophet	Meta的预测库
可视化	Plotly + WordCloud	交互式图表

三、功能模块详细设计

3.1 舆情监测模块

功能需求

- ☑ 关键词设置与保存
- ☑ 定时数据采集
- ☑ 数据来源：新闻网站 + 微博
- ☑ 数据去重与过滤
- ☑ 实时监控状态展示

数据采集方案

```
**爬虫**
```python
# 目标网站:
1. 新浪新闻 (https://news.sina.com.cn/)
2. 搜狐新闻 (https://news.sohu.com/)
3. 微博搜索 (https://s.weibo.com/)
```
```

数据存储结构

```
-- 舆情数据表

-- 监控配置表
```

3.2 舆情分析模块

功能需求

- ☑ 批量情感分析
- ☑ 情感统计（正面/负面/中性比例）
- ☑ 关键词提取与词云
- ☑ 时间趋势分析
- ☑ 热点话题排行

情感分析方案

方案A：传统NLP方法（基础用于对比）

```
# SnowNLP - 轻量级中文情感分析
from snownlp import SnowNLP

def analyze_sentiment(text):
    s = SnowNLP(text)
    score = s.sentiments  # 0-1之间，越接近1越正面
    if score > 0.6:
        return 'positive', score
    elif score < 0.4:
        return 'negative', score
    else:
        return 'neutral', score
```

方案B：LLM增强（本项目）

```
# 使用OpenAI兼容API
from openai import OpenAI

def analyze_with_llm(text):
    client = OpenAI()
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "分析以下文本的情感倾向"},
            {"role": "user", "content": text}
        ]
    )
    return response.choices[0].message.content
```

可视化方案

```
# 1. 饼图
# 2. 柱状图
...
```

3.3 舆情预测模块

功能需求

- ☑ 基于历史数据的时间序列预测
- ☑ 预测未来3-7天趋势
- ☑ LLM辅助趋势分析
- ☑ 预测结果可视化